Chapter 15.3: Files and I/O

About this page:

Key questions: How do I read data from outside the program?

Topics: Learn about the Java I/O library

What will I do? Reading and programming.

Indicative difficulty level: Intermediate - lot of new stuff.

Rough estimate of workload? 3-4 hours

Point value: S130

Related projects: Kalevala (https://gitmanager.cs.aalto.fi/static/studio2_k2022dev2-horrible-gitmanager-interface/projects/given/Kalevala/), WeatherStation (exercise in maintenance, opens soon).

Streams

I / O Streams are structures that are often used for transmitting data between the program and the outside world. From the program point of view, streams are serial data. The input stream is a sequence of characters that the program can read at its own pace. Correspondingly, the outgoing stream is a structure into which the program can write characters, and thereafter something happens to it. The data stream is an abstraction of what is on the other end of the stream. It hides special features of the underlying target, providing a small set of basic features that can be used to write or read data in the stream. The other end of the stream is often a file on a disk, a network connection, a keyboard, etc. At a minimum, the data stream is a class with a single method that reads a character or returns an error code that tells that the stream is closed.

Data stream is not a Scala or Java-specific abstraction, but something that can be found in almost every programming language in some form.

Data streams in Java

Scala's own support for data communication is limited to the class `scala.io.Source`, which describes a data source and which can be used to retrieve a number of different data streams. In practice, it is a wrapper class that offers a more "Scala-like" interface to Java I/O classes. In the exercises of this section, we use Java classes for reading information, just for training to use them.

Java splits low-level data streams into byte handling `java.io.InputStream` and `java.io.OutputStream`, as well as string managing streams `java.io.Reader` and `java.io.Writer`. More advanced streams with more features have been derived from these.

Outputting bytes: OutputStream

`java.io.OutputStream` objects can be used to write into data streams in which the data consists of bytes. Different versions of the method `write` can either write a single byte into the data stream or alternatively write a set of bytes from an array.

When writing to the data stream is ultimately terminated, the data stream must be closed by invoking the method `close()` so that the reserved system resources are released and the other end of the data stream can be informed about finishing the data transfer.

```
/*
  * Prints a single Byte into a stream. Int parameter bits that do not
  * fit into one byte are ignored
  */
def write(b: Int): Unit

/*
  * Prints all bytes in the parameter array into the stream.
  */
def write(b: Array[Byte]): Unit

/*
  * As above, but only part of the array is printed.
  */
def write(b: Array[Byte], offset: Int, length: Int): Unit

/*
  * Closes the output stream and releases the reserved resources
  * (for example, buffer tables, etc.)
  */
def close(): Unit
```

Simply writing into the data stream is not a guarantee that the written data is immediately transferred to the data stream. In order to make output more efficient, *buffers* are often used to join a number of small output operations to larger but less frequently executed operations. In buffered streams, the forward transfer of data must be separately commanded by calling the method `flush`. Some `OutputStream` subclasses automatically flush the buffer.

```
/*
  * The Flush method forces the buffer of the stream to be written into the stream.
  * Often close() performs a flush() command before closing the stream.
  */
def flush(): Unit
```

Reading bytes: InputStream

`java.io.InputStream` is the counterpart to `OutputStream`. Its class entities are used when you want to read bytes from a data stream. Like the corresponding writing methods, it is possible to read a single byte or a set of bytes from the `InputStream` into a designated array.

```
// Reads a single Byte from the stream, returning it as an Int
def read(): Int

// Reads a number of bytes and stores them into an array given as a parameter.
// Returns how many bytes were read.
def read(b: Array[Byte]): Int

// Reads a number of bytes and stores them in the desired location in the array.
// Returns how many bytes were read.
// If the method returns -1, then the stream was terminated and there is no data left.
def read(b: Array[Byte], off: Int, len: Int): Int

// Like output streams, input streams can have resources,
// which are released when the stream is closed.
// Closing the streams at the end of the tasks is important.
def close(): Unit

// Instead of reading, we can deliberately ignore legible characters
// with a skip method. Skip ignores up to n characters. (see the return value)
def skip(n: Long): Long
```

More about reading

As we mentioned, some methods return a number that tells how many characters were read. For example, when reading a file from a disk, the program can easily be faster than the disk. In this case, the method tries to read at least one character. But what happens if no characters could still be read, but the stream is not closed?

If the data stream does not have any readable data, the execution of the program will wait for the data to be available. This waiting is called ”blocking”. While waiting, the program hardly consumes any CPU power, but it is not able to do anything else on the other hand.

Blocking ends either when the method gets something to read or the readable data stream is closed.

A larger example

Low level I/O requires some handwork because at this level there is no method that can read everything from a stream into a buffer. Such an implementation might look like this. (Note the do-while loop, where the read operation is repeated. The cursor shifts based on the number of read characters, informed by the read operation.)

```
import java.io._

object Reading extends App {

  def readFromStream(in: InputStream): Array[Byte] = {
    var cursor = 0
    var readBytes = 0

    /*
      * Constant size buffer for readable data.
      * (Bad solution but fits on this page)
      */
    val buffer: Array[Byte] = Array.ofDim[Byte](65536)

    /*
      * Tries to read a maximum of 1024 characters at a time.
      * If the stream is closed, the read operation returns -1.
      * Otherwise, we will get a number from 1 to 1024, which will tell you how many characters
      * the read operation was able to read this time until the buffer was empty.
      *
      * In the loop, the cursor is moved forward corresponding to the read data.
      */
    do {
      readBytes = in.read(buffer, cursor, 1024)

      if (readBytes != -1)
        cursor += readBytes
    } while (readBytes != -1)

    /*
      * Finally, ensure that the stream is closed and the data is copied
      * to a correctly sized array, which is then returned.
      */

    in.close()
    val c: Array[Byte] = Array.ofDim[Byte](cursor)
    Array.copy(buffer, 0, c, 0, cursor)

    c
  }
}
```

Derivatives, PrintStream

Low level I/O requires work and is error-prone. Plain byte streams are often not used as such, but instead we use some of their derivatives which have more stream functions. Scala's `Console.println` directly invokes the Java `PrintStream` class method `println`. `PrintStream` adds

`print` and `println` methods to the previously mentioned `OutputStream` class. These streams can output string expressions from all elementary types, strings, and other objects (printing values returned by their `toString()` methods).

String streams

The String streams `Reader` and `Writer` differ from the byte streams in that their data consists of characters (char) instead of bytes. Java and Scala use 16-bit Unicode characters, `Char`, while `Byte` is only an 8-bit long data type. In practice, string streams are used to transfer data when it is human readable, and byte streams if the result is read only by a machine.

`Reader` and `Writer`, as well as their derivatives, are useful for handling textual data. Classes `InputStreamReader` and `OutputStreamWriter` provide an interface that combines byte streams and string streams.

```
// let us change System.in to a string stream
val isr: Reader = new InputStreamReader(System.in)

// convert the stream to be buffered
val lineReader = new BufferedReader(isr)

// read a line of text from the stream
var inputLine = lineReader.readLine()
```

In the example above, the data stream `System.in` from the keyboard is converted to a format that can be read line by line. Note in particular how streams are connected sequentially. The example class `BufferedReader` allows you to read the text by lines, but its main function is to make reading efficient by using buffers.

> **Unicode and bytes**
>
> 16 bit Unicode is not quite so simple as it might seem. However, it is beyond the scope of the course to discuss its challenges. Interested readers could view, for example, Tim Bray's essay on the topic. (https://www.tbray.org/ongoing/When/200x/2003/04/26/UTF)

BufferedWriter, BufferedOutputStream

The classes `BufferedWriter` and `BufferedOutputStream` join operations together and thus reduce the underlying data stream events. When small write operations are applied to the buffered stream, it gathers the written data into the buffer, from which it is then moved to the underlying stream. This usually happens only when the buffer is full.

A historical anecdote: Before the time of SSDs (solid-state drive), it was possible to hear a program writing data on a hard drive, as the hard drive's writing head moved with each character. It is both more efficient and physically less burdening to the machine to carry out writing in larger batches.

If desired, the user can force to empty the writing buffer by invoking the method `flush()`. This is especially important when the file is being closed and you want to make sure that characters are transferred to a disk or into bidirectional communication over the network so that data does not remain in the buffer simply because the message is too short.

BufferedReader, BufferedInputStream

Likewise, the input streams try to read larger blocks of data at a time into the buffer, so that small operations targeted to the buffer do not cause a large number of events in the underlying stream.

I/O and Exceptions

Almost any method and constructor associated with data streams can throw the exception `IOException`. Therefore, almost all operations on data streams are carried out within a `try-catch` block.

```
// Example, reading from a keyboard via a lower level API

import java.io._

object Example extends App {

  val converter = new InputStreamReader(System.in)
  val lineReader = new BufferedReader(converter)

  try {
    var inputLine = lineReader.readLine()

    while(inputLine != null && inputLine != "exit") {
      scala.Console.println(inputLine.toUpperCase)
      inputLine = lineReader.readLine()
    }
  } catch {
    case e: IOException =>
      scala.Console.println("Reading finished with an error")
  }
}
```

Files

Reading and writing files is often performed through ”file streams”.

- `FileInputStream(String name) throws FileNotFoundException`
- `FileOutputStream(String name) throws IOException`

There are corresponding classes for string files

- `FileWriter`

- `FileReader`

When processing files, closing data streams is especially important because open files can prevent reading or writing a file from other programs.

Example, Reading a file

Possible exceptions can be handled either in separate `try` blocks ...

```
import java.io._

object FileExample extends App {

 def readMyFile(sourceFile: String):Unit = {

   val myFileReader = try {
      new FileReader(sourceFile);
   } catch {
     case e: FileNotFoundException =>
       println("File not found")
      return
   }

   val lineReader = new BufferedReader(myFileReader)

   try {
      var inputLine = lineReader.readLine()

      while (inputLine != null) {
        println(inputLine)
        inputLine = lineReader.readLine()
      }
    } catch {
      case e: IOException =>
        println("Reading finished with error")
    }
  }

 readMyFile("testfile.txt")

}
```

... or within one large block. Notice how the inner try-final structure ensures data stream closure.

```
try {
  // opens an incoming character stream from the file
  val fileIn = new FileReader("example.txt")

  // Creates a buffered stream with which it is possible to
  // read line by line from the previous stream

  val linesIn = new BufferedReader(fileIn)

  // At this point, the streams should be open
  // so we must remember to close them.
  try {

    // Read the text from the stream line by line until the read line
    // is null. Then we know that
    // the stream (and also the file) end has been reached.

      var oneLine = linesIn.readLine()

      while (oneLine != null) {
          System.out.println(Oneline)
          oneLine = linesIn.readLine()
        }
    } finally {
      // Close open streams
      // This will be executed if the file has been opened
      // regardless of whether or not there were any exceptions.

      fileIn.close()
      linesIn.close()
    }
  } catch {
    case notFound: FileNotFoundException =>
    // Response here to a failed file opening.
    case e: IOException =>
    // Response here to unsuccessful reading
}
```

File

`java.io.File` class objects describe file system files. In practice, `File` describes a file path through which you can view and change file properties. However, the path in the `File` object does not need to point to an existing file.

File system directories are also files and are also described with `File` class objects. The `File` object can be used, for example, to create, delete and rename files and directories, and check their existence, permissions, and sizes. A file listing can be requested from the `File` Object describing the directory. For more information about the `File` class' documentation, see Java documentation (http://docs.oracle.com/javase/7/docs/api/java/io/File.html)

Example: own "ls" command

```
import java.io._

object myLs extends App {

  val currentDirectory = new File("./")
  val listing = currentDirectory.listFiles()

  for (oneFile <- listing) {
    if (oneFile.isFile) {
      println(oneFile.length + " bytes\t " + oneFile.getName)
    }
  }
}
```

Now we are ready for two exercises, where you can train what you learned above.

3. Kalevala (70p)

In this exercise, you will practice the basics of file handling. You will be handling a part of a Finnish poem from the Finnish folk lore epic Kalevala.

We will be using the `BufferedReader` and the `FileReader` classes from the `java.io` package to read the poem line by line. When reading a file this way, it is possible to choose some parts of the file and leave some parts out. In the exercise you will be making different filters to choose only particular lines of the poem in each part.

Structure of the Poem text file (kalevala3.txt)

In the exercise, there is included the text file of the third poem of the Kalevala epic (kalevala3.txt). The poem consists of lines of the poem separated into verses with an empty line. Thus, from the empty lines you will know that a new verse will start on the following line. The example below demonstrates the structure:

```
    Verse number one
    goes like this

    verse number two
    has more lines
    but is still
    a verse, too

    This is
    the third verse
```

The Kalevala poems have some other structural features as well, but for the purposes of the exercise we will only consider the lines and verses.

Reading the file

In the exercise, reading the file is separated into two parts. The method `readFile` should use the `FileReader` and `BufferedReader` to create the buffered output stream for the file (aka a `BufferedReader` object).

The actual reading through the file line by line will then be done in separate filter methods that the `readFile` function takes as parameters. The overall functionality will be quite similar to the `FileReader` example above except that reading line-by-line is done inside a separate method. More on this in the Filtering section.

Remember to check for the `FileNotFoundException` and `IOException` as described in the examples above. Remember also in the end to close both reader streams with the `close()` method. You can use the `try-catch` structure to check for the exceptions, as described in the examples.

Filtering the file

In the exercise, we want to design filters that will extract only certain lines from the poem and save the into a string sequence Seq[String]. The filter methods takes the `BufferedWriter` as a parameter which can be used to read through a file line by line. You can take a look at the `noFilter` method to get an idea how the filer methods are supposed to traverse through the lines of the file.

The filters in the exercise will be, as follows.

`noFilter` – An example "filter" that does no filtering, but simply reads through the whole file and saves every line to a string sequence.

`firstLineOfEachVerse` – A filter that only saves the first line of each verse.

`rowsWithVainamoinen` – Keeps only the rows that have the word "Väinämöinen" in them.

`EveryOtherRowOfVerses` – Filters out every other row of each verse. This filter should keep the empty lines between the verses. For each verse, it should first save the first line of the verse, and then only take every other line into the string sequence.

Writing the results to a new file

Finally, once your filters are working correctly, the `writeToFile` method should write the results of your file reading and filtering to a new file. Here we use `FileWriter` and `BufferedWriter`.

The `writeToFile` takes a string sequence as a parameter, which corresponds to the sequence of the filtered lines produced by the filters. This method should simply write the contents of the string sequence to a designated file location.

Again, remember to check for the `FileNotFoundException` and `IOException` with the writer object. Remember also in the end to close both writer streams with the `close()` method. You can use the `try-catch` structure to check for the exceptions.

Exercise 1 (Kalevala) The deadline has been archived (Thursday, 1 June 2023, 12:00).

Select your files for grading

Show anyway

KalevalaReader.scala

Choose File No file chosen

Submit

4. Weather Station (60p)

In the next exercise you will handle data points created by a Weather Station. The data consists of different weather-related parameters and your task is 1) to read data points out from a file and 2) to write a given data point in to a file while preserving the Weather Data file format.

The structure of the data in the file is separated into blocks that start with a ”#” symbol and a new block always starts from a new line.

The data blocks that are included in the Weather Data file are, as follows:

Data type	Header	Length	Unit
Version data	CLI	3	Version no.
Date	DAY	8	DDMMYYYY
Time stamp	TIM	6	HHMMSS
Air Pressure	APR	4	hPA
Temperature	TMP	3	Celsius
Humidity	HUM	3	%
Precipitation	PRC	3	mm

The Data type tells you what type of data the data block represents. There are corresponding variables for each of the Data Types in the WeatherDataPoint class except for the Version data.

There is a unique three character header associated with each data block. Each header is proceeded with the symbol ”#” and the next data block always starts from a new line and with a new header.

The length tells you how many characters there are after the header that represent the actual data of the block. So Time stamp, for example, always has 6 digits in its representation (the DDMMYYYY format). The unused spaces in the front will be filled with extra zeros (0) e.g. if the data block has length 3 and the data input is 10 (only two digits) the resulting data block should be ”010”.

A sample file would look something like this:

```
    #CLI012
    #DAY01012020
    #TIM000300
    #TMP010
    #APR1013
    #HUM050
    #PRC000
```

In the first part, you will read through a weather data file and return a corresponding WeatherDataPoint object that will hold the information. The WeatherDataPoint looks like this:

```
case class WeatherDataPoint (
  date: Int,
  timestamp: Int,
  temperature: Int,
  pressure: Int,
  humidity: Int,
  precipitation: Int
)
```

We will use `FileReader` and `BufferedReader` to read through the file line by line (see the examples above).

In the second part you will be given a WeatherDataPoint object which you need to convert into the WeatherStation file form. You should keep the order of the blocks as they are represented in the chart above. In this part you should consider how to handle the extra zeroes that must be added in the beginning of smaller valued data points .

⚠ This course has been archived (Thursday, 1 June 2023, 12:00).

Unfortunately, this assignment is currently under maintenance.

Networking

The `Java.net` API offers two ways to transfer information over the network.

- UDP (based on sending individual packets)
- TCP (packet switching data stream)

The TCP protocol hides packets from the user and provides the connection between endpoints, similarly to the data streams as we have just discussed.

In practice, the data stream is accessed by using the class `Socket`, which in Java has both an `OutputStream` and an `InputStream`.

Scala has its own class `scala.io.Source` for reading files and data streams. Below there is an example for using it to read a web page.

```
import scala.io.Source

object ReadWWW extends App {

  try {

    val urlStream = Source.fromURL("https://www.aalto.fi/en/")
    urlStream.getLines().foreach(println)

  } catch {
    case e: UnknownHostException =>
      System.out.println("Address Wrong")
    case e: IOException =>
      System.out.println("Something wrong")
  }
}
```

Summary

- We learned about the Java I/O library.
- We learned more about how to manage stream and how to connect them to files.
- Binary and string streams are managed with different classes.
- Exception handling is a normal practice when handling files.

Feedback

Please note that this section must be completed individually. Even if you worked on this chapter with a pair, each of you should submit the form separately

Loading assignment...

« Chapter 15.2: Exceptions and attempts (https://pl…

Round 16 » (https://plus.cs.aalto.fi/studio_2/k2022/…